СИСТЕМ БА СҮЛЖЭЭНИЙ УДИРДЛАГА

ХИЧЭЭЛ 1

Орчин үеийн систем ба сүлжээний удирдлага: үүрэг, lifecycle, automation-first, security-by-design

Уламжлалт операторчлолоос инженерчлэл рүү

Хугацаа: 2 цаг лекц + 2 цаг лаборатори · Урьдчилсан мэдлэг: OS ба сүлжээний үндэс

Хэрэглэх хэрэгсэл: Linux, Git, Ansible, Terraform, Docker, Prometheus

Энэ лекц юуны тухай вэ

Системийн удирдлагын мэргэжил сүүлийн арван таван жилд эрс өөрчлөгдсөн. Нэг сервер рүү SSH-ээр нэвтэрч гараар тохируулдаг байсан ажил өнөөдөр **кодоор тодорхойлогдсон дэд бүтэц**, автомат байршуулалт, тасралтгүй хяналт болж хувирсан.

Энэ лекцээр мэргэжлийн **үүргийн газрын зураг**, системийн **амьдралын мөчлөг**, орчин үеийн хоёр үндсэн зарчим болох **automation-first** ба **security-by-design**-ыг судална.

БАГШИД ЗОРИУЛСАН ТЭМДЭГЛЭЛ

Энэ лекц нь хэрэгслийн заавар биш — **зарчмын** лекц. Terraform, Ansible, Kubernetes-ийг дэлгэрэнгүй үзэхгүй, зөвхөн тэдгээр ямар асуудлыг шийддэгийг тайлбарлана. Хэрэгслийн гүнзгий агуулга дараагийн лекцүүдэд.

ЗОРИЛГО

Суралцахуйн үр дүн

- Систем ба сүлжээний удирдлагын **үүргүүдийг** (SysAdmin, NetAdmin, SRE, DevOps, Platform, SecOps) ялган тайлбарлах
- Үйлчилгээний **амьдралын мөчлөгийн** долоон үе шатыг нэрлэж, тус бүрийн үр дүнг тодорхойлох
- **SLI, SLO, SLA, error budget** ойлголтуудыг ялгаж, бэлэн байдлын тоог хугацаа болгон хөрвүүлэх
- **Toil**-ыг таньж, автоматжуулалтын шатлалын дагуу бууруулах төлөвлөгөө гаргах
- **IaC**, тохиргооны удирдлага, **idempotency**, GitOps-ийн зарчмыг тайлбарлах
- **Least privilege, defense in depth, zero trust, secure defaults** зарчмуудыг хэрэглэх
- Нууц мэдээллийн удирдлага, залруулгын менежмент, аудитын бүртгэлийн шаардлагыг тодорхойлох
- Инцидент, өөрчлөлт, асуудлын менежментийг ялгаж, **blameless postmortem** бичих

АГУУЛГА

Лекцийн бүтэц

Хэсэг	Агуулга	Ойролцоо цаг
I	Мэргэжлийн үүрэг: SysAdmin, NetAdmin, SRE, DevOps, SecOps	20 мин
II	Үйлчилгээний амьдралын мөчлөг, SLO, инцидент, backup/DR	30 мин
III	Automation-first: toil, IaC, idempotency, GitOps, observability	30 мин
IV	Security-by-design: CIA, least privilege, zero trust, secrets	30 мин
V	Практик: лавлах архитектур, runbook, DORA хэмжүүр, нэгтгэл	10 мин

ХЭСЭГ I

Мэргэжлийн үүрэг

Системийн удирдлага гэж юу вэ, уламжлалт ба орчин үеийн ажлын ялгаа, үүргүүдийн газрын зураг.

Системийн удирдлага гэж юу вэ

Систем ба сүлжээний удирдлага гэдэг нь байгууллагын мэдээллийн технологийн үйлчилгээг найдвартай, аюулгүй, хэмжигдэхүйц чанартайгаар **тасралтгүй ажиллуулах** инженерийн салбар юм.

Тодорхойлолтын гурван түлхүүр үг.

1. **Үйлчилгээ**

   Сервер, сүлжээ бол зорилго биш — **хэрэглэгчид үзүүлж буй үйлчилгээ** л зорилго. Сервер асаалттай байх нь үйлчилгээ ажиллаж байна гэсэн үг биш.

2. **Тасралтгүй**

   Ажиллаж эхлүүлэх нь ажлын 20%. Үлдсэн 80% нь өөрчлөлт, ачаалал, эвдрэлийн дунд **ажиллуулсаар байх** явдал.

3. **Хэмжигдэхүйц**

   «Сайн ажиллаж байна» гэдэг үнэлгээ биш. Бэлэн байдал, хариу өгөх хугацаа, алдааны хувь — тоогоор илэрхийлэгдэнэ.

Уламжлалт ба орчин үеийн ажлын харьцуулалт

Шинж	Уламжлалт (2005 орчим)	Орчин үеийн (өнөөдөр)
Тохиргоо	SSH-ээр гараар, сервер бүрд	Кодоор тодорхойлж, автоматаар
Серверийн загвар	Тэжээвэр амьтан (pet) — нэр өгч эмчилнэ	Мал сүрэг (cattle) — эвдэрвэл солино
Баримтжуулалт	Wiki дээрх заавар (хуучирдаг)	Код өөрөө баримт (Git түүхтэй)
Байршуулалт	Шөнийн цонх, гараар, эрсдэлтэй	Өдөрт олон удаа, автомат, буцаагдах
Хяналт	«Унасан уу?» гэсэн шалгалт	Метрик, лог, trace — шалтгаан хүртэл
Аюулгүй байдал	Эцэст нь нэмдэг давхарга	Загварын үе шатнаас эхэлдэг
Алдааны хандлага	Хэн буруутайг олох	Системийн шалтгааныг олох

Үүргүүдийн газрын зураг

Үүрэг	Гол хариуцлага	Түлхүүр хэмжүүр
System Administrator	Сервер, УС, үйлчилгээ, хэрэглэгч, нөөцлөлт	Бэлэн байдал, тасалдлын хугацаа
Network Administrator	Сүлжээний бүтэц, чиглүүлэлт, VLAN, VPN, галт хана	Багц алдагдал, саатал, зурвасын өргөн
Site Reliability Engineer	SLO, инцидент, capacity, toil бууруулах	Error budget, MTTR
DevOps Engineer	CI/CD, байршуулалтын урсгал, хөгжүүлэгчийн туршлага	Байршуулалтын давтамж, lead time
Platform Engineer	Дотоод платформ, self-service хэрэгсэл	Хөгжүүлэгчийн хүлээх хугацаа
Security Engineer	Хамгаалалт, эрх, аудит, эмзэг байдал	Засварын хугацаа, илрүүлэлт
Cloud Engineer	Үүлэн нөөц, өртөг, олон бүсийн архитектур	Өртөг/нэгж, бэлэн байдал

SRE ба DevOps — юугаараа ялгаатай вэ

Хоёр нэр томьёог хольж хэрэглэх нь түгээмэл. Гэвч эдгээр нь **өөр асуултад** хариулдаг.

1. **DevOps — соёл ба урсгал**
   «Хөгжүүлэлт ба ажиллагааны хооронд байгаа хана яаж арилгах вэ?» Хариулт: хамтарсан хариуцлага, автомат урсгал, богино цикл.
2. **SRE — хэрэгжүүлэлт ба хэмжүүр**
   «Найдвартай байдлыг яаж **тоогоор** удирдах вэ?» Хариулт: SLO тогтоож, error budget-ээр шийдвэр гаргана.

> ***«SRE бол DevOps-ийг хэрэгжүүлэх нэг тодорхой арга» — салбарт өргөн хүлээн зөвшөөрөгдсөн тодорхойлолт.***

Хариуцлагын хил хязгаарыг тодорхойлох

Тасалдлын үед хамгийн их цаг алддаг зүйл нь техникийн асуудал биш, **«хэн хариуцах вэ?»** гэсэн эргэлзээ юм. Үүнийг урьдчилан бичгээр тогтооно.

Давхарга	Хариуцагч	Жишээ асуудал
Дэд бүтэц	Cloud / Infra баг	Виртуал машин ачаалахгүй байна
Сүлжээ	Network баг	Бүсүүд хооронд багц алдагдаж байна
Платформ	Platform баг	Kubernetes кластер шинэ pod авахгүй
Үйлчилгээ	Бүтээгдэхүүний баг	API 500 алдаа буцааж байна
Өгөгдөл	DBA / Data баг	Хуулбарлалт хоцорч байна
Аюулгүй байдал	Security баг	Сэжигтэй нэвтрэлт илэрсэн

Хилийн бүсэд (жишээ нь «сүлжээ үү, програм уу?») хамтарсан протокол урьдчилан тогтоосон байх ёстой.

Өдрийн ажлын бодит бүтэц

Мэргэжлийн ажлын цаг хэрхэн хуваарилагддаг талаар салбарын судалгаанууд ойролцоо дараах зургийг харуулдаг.

Ажлын төрөл	Эрүүл багт	Асуудалтай багт
Инженерийн ажил (автоматжуулалт, сайжруулалт)	50%+	10–20%
Toil (давтагдах гар ажил)	< 30%	40–60%
Инцидент, тасалдал	10%	20–30%
Уулзалт, зохицуулалт	10–15%	10–15%

ГОЛ ДҮРЭМ

SRE-ийн практикт **toil 50%-иас хэтэрвэл** тухайн багт ажлын ачааллыг бууруулах эсвэл автоматжуулалтад цаг гаргах шийдвэр гаргадаг. Энэ бол зөвлөмж биш, удирдлагын дүрэм.

Toil гэж юу вэ

Toil гэдэг нь үйлчилгээг ажиллуулахтай холбоотой, **давтагдах, гараар хийгддэг, автоматжуулж болох, урт хугацааны үнэ цэн үүсгэдэггүй** ажил юм.

Ажил toil мөн эсэхийг дараах зургаан шинжээр шалгана.

- **Гараар** хийгддэг — хүн гар оролцоо шаардана
- **Давтагдана** — өмнө нь олон удаа хийсэн, дахин хийнэ
- **Автоматжуулж болно** — машинаар орлуулах боломжтой
- **Тактикийн** — урьдчилан төлөвлөсөн биш, хариу үйлдэл
- **Урт хугацааны үнэ цэнгүй** — дууссаны дараа систем сайжирдаггүй
- **Үйлчилгээний өсөлттэй шугаман өснө** — хэрэглэгч 2 дахин өсвөл ажил 2 дахин өснө

Toil мөн үү, биш үү

Ажил	Toil мөн үү	Тайлбар
Хэрэглэгчийн нууц үг сэргээх	Тийм	Self-service болгож автоматжуулна
Диск дүүрэхэд гараар цэвэрлэх	Тийм	Ротаци ба автомат цэвэрлэгээ шаардлагатай
Шинэ архитектур зохиох	Үгүй	Урт хугацааны үнэ цэн үүсгэнэ
Postmortem бичих	Үгүй	Дахин давтагдахаас сэргийлнэ
Сервер бүрд гараар нөхөөс тавих	Тийм	Тохиргооны удирдлагаар шийднэ
Хяналтын дүрэм сайжруулах	Үгүй	Ирээдүйн ажлыг бууруулна
Тайлан гараар бэлдэх	Тийм	Автомат тайлангаар солино

Ур чадварын үндсэн багц

Бүлэг	Агуулга
Үйлдлийн систем	Linux дотоод бүтэц, процесс, санах ой, файлын систем, systemd
Сүлжээ	TCP/IP, DNS, HTTP, TLS, чиглүүлэлт, NAT, галт хана, оношилгоо
Скрипт ба хэл	Shell, Python, YAML, JSON, regex
Дэд бүтцийн код	Git, Terraform, Ansible, container, Kubernetes
Хяналт	Метрик, лог, trace, dashboard, alert зохиох
Аюулгүй байдал	Эрхийн загвар, нууц удирдлага, hardening, эмзэг байдал
Өгөгдөл	Нөөцлөлт, сэргээлт, хуулбарлалт, RPO/RTO
Хүний чадвар	Баримтжуулалт, инцидентийн харилцаа, тайлбарлах ур чадвар

Хэсэг I-ийн гол санаа

> ***Орчин үеийн системийн администратор бол системийг ЗАСВАРЛАДАГ хүн биш, засварлах шаардлагагүй систем ЗОХИОН БАЙГУУЛДАГ инженер юм.***

Энэ шилжилтийн практик утга: гараар давтан хийж буй ажил бүр нь **ирээдүйн автоматжуулалтын жагсаалт** гэж үзэгдэнэ.

ХЭСЭГ II

Үйлчилгээний амьдралын мөчлөг

Төлөвлөлтөөс ашиглалтаас гаргах хүртэл: долоон үе шат, SLO, өөрчлөлт ба инцидентийн менежмент, нөөцлөлт.

Долоон үе шат

1 **Plan — Төлөвлөх**

Шаардлага, багтаамжийн тооцоо, өртөг, эрсдэлийн үнэлгээ, SLO-ийн төсөл.

2 **Design — Зохиох**

Архитектур, сүлжээний бүтэц, эрхийн загвар, аюулгүй байдлын хяналт, DR стратеги.

3 **Build — Барих**

IaC бичих, golden image бэлдэх, CI/CD урсгал, тест орчин.

4 **Deploy — Байршуулах**

Үе шаттай гаргалт (canary, blue-green), буцаах төлөвлөгөө, шалгалт.

5 **Operate — Ашиглах**

Хяналт, дуудлага, инцидент, өөрчлөлт, багтаамжийн удирдлага.

6 **Improve — Сайжруулах**

Postmortem, toil бууруулах, гүйцэтгэл ба өртгийн оновчлол.

7 **Decommission — Ашиглалтаас гаргах**

Өгөгдлийн шилжүүлэлт, найдвартай устгал, эрх хураах, баримтжуулалт.

Багтаамжийн төлөвлөлт

Багтаамжийн төлөвлөлт нь «хэдэн сервер хэрэгтэй вэ?» гэсэн асуултад **тоо баримтаар** хариулах үйл явц.

1. **Одоогийн ачааллыг хэмжих**

   Хүсэлт/сек, өгөгдлийн өсөлт, оргил цагийн харьцаа (peak-to-average).

2. **Өсөлтийг таамаглах**

   Түүхэн өгөгдөл + бизнесийн төлөвлөгөө. Улирлын хэлбэлзлийг тооцно.

3. **Толгойн зайг тодорхойлох**

   Ердийн ачааллын дээр 30–50% нөөц. Нэг бүс унахад бусад нь даах чадвартай эсэх.

4. **Ачааллын тест хийх**

   Онолын тооцоог бодит тестээр батална. Хаана эхлээд саатал үүсэхийг ол.

ТҮГЭЭМЭЛ АЛДАА

Дундаж ачаалалд тулгуурлан төлөвлөх. Систем **оргил** ачааллыг даах ёстой — дундаж нь 20%, оргил нь 80% ачаалалтай байж болно.

SLI, SLO, SLA — гурван нэр томьёо

Нэр	Утга	Жишээ
SLI	Хэмжигдэж буй бодит үзүүлэлт	Амжилттай HTTP хүсэлтийн эзлэх хувь
SLO	Дотооддоо тогтоосон зорилт	30 хоногт 99.9% амжилттай байх
SLA	Хэрэглэгчтэй байгуулсан гэрээ	99.5%-аас доош бол төлбөр буцаана

ХАМААРАЛ

SLA нь SLO-оос сул, SLO нь SLI-д тулгуурлана. Хэрэв SLA 99.5% бол дотоод SLO-г 99.9% тогтооно — гэрээ зөрчихөөс өмнө анхааруулга авахын тулд.

Бэлэн байдлын тоо — хугацаа болгон

Бэлэн байдал	Жилд тасалдал	Сард	7 хоногт
99%	3.65 өдөр	7.31 цаг	1.68 цаг
99.9%	8.77 цаг	43.8 мин	10.1 мин
99.95%	4.38 цаг	21.9 мин	5.04 мин
99.99%	52.6 мин	4.38 мин	1.01 мин
99.999%	5.26 мин	26.3 сек	6.05 сек

ПРАКТИК УТГА

99.99% гэдэг нь сард 4 минут л тасалдал. Ердийн байршуулалт 2 минут болдог бол сард **хоёр удаа** байршуулаад л төсөв дуусна. Тиймээс өндөр SLO нь автомат, тасалдалгүй байршуулалтыг **шаарддаг**.

Error budget — алдааны төсөв

Error budget гэдэг нь SLO-оос үлдэх зөвшөөрөгдөх алдааны хэмжээ юм: `100% − SLO`.

SLO нь 99.9% бол 30 хоногийн төсөв нь **43.8 минут**. Энэ бол алдаа биш — **зарцуулж болох нөөц**.

1. **Төсөв үлдсэн байвал**
   Шинэ функц гаргах, эрсдэлтэй туршилт хийх, байршуулалтыг хурдасгах боломжтой.
2. **Төсөв дууссан бол**
   Шинэ функцийн гаргалтыг зогсоож, найдвартай байдлын ажилд бүх нөөцийг чиглүүлнэ.

Error budget нь «хурд vs найдвартай байдал» гэсэн мөнхийн маргааныг тоон шийдвэр болгон хувиргадаг.

Сайн SLI-г хэрхэн сонгох вэ

Муу SLI	Сайн SLI	Шалтгаан
CPU ачаалал < 80%	Хүсэлтийн 99% нь 300мс дотор	Хэрэглэгч CPU-г мэдрэхгүй
Сервер ping хариулж байна	Нэвтрэлт амжилттай болсон хувь	Ping нь үйлчилгээ ажиллаж байгааг батлахгүй
Диск дүүрээгүй	Захиалга амжилттай хадгалагдсан хувь	Хэрэглэгчийн зорилготой холбоотой
Дундаж хариу хугацаа	p95 / p99 хариу хугацаа	Дундаж нь удаан хэрэглэгчийг нуудаг

ДҮРЭМ

SLI нь **хэрэглэгчийн туршлагыг** хэмжих ёстой, дэд бүтцийн төлөвийг биш. «Хэрэглэгч аз жаргалтай байна уу?» гэсэн асуултад хамгийн ойр байх үзүүлэлтийг сонгоно.

Өөрчлөлтийн менежмент

Тасалдлын дийлэнх хувь нь **саяхан хийсэн өөрчлөлтөөс** үүсдэг. Тиймээс өөрчлөлтийг ангилж, эрсдэлд тохирсон процесс хэрэглэнэ.

Төрөл	Тодорхойлолт	Батлах журам
Standard	Урьдчилан батлагдсан, давтагдах, эрсдэл бага	Батлах шаардлагагүй — автомат
Normal	Төлөвлөсөн, үнэлгээ шаардсан	Хянан батлах (CAB эсвэл багийн хяналт)
Emergency	Тасалдал зогсоох яаралтай засвар	Дараа нь баримтжуулж батлуулна

ОРЧИН ҮЕИЙН ХАНДЛАГА

Хүнд журам нь өөрчлөлтийг **томруулж, ховордуулж, улмаар илүү эрсдэлтэй** болгодог. Тиймээс жижиг, байнгын, автомат шалгалттай өөрчлөлтийг илүүд үзнэ.

Байршуулалтын аюулгүй хэв загварууд

Хэв загвар	Зарчим	Давуу тал
Blue-green	Хоёр ижил орчин, урсгалыг шилжүүлнэ	Буцаалт агшин зуур
Canary	Эхлээд 1–5% урсгал, дараа нь өргөжүүлнэ	Асуудал цөөн хэрэглэгчид нөлөөлнө
Rolling	Нэг нэгээр нь шинэчилнэ	Нэмэлт нөөц бага шаардана
Feature flag	Код гаргасан ч функцийг тусад нь асаана	Байршуулалт ба гаргалтыг салгана

ЗААВАЛ БАЙХ ЗҮЙЛ

Байршуулалт бүр **буцаах төлөвлөгөөтэй** байна. «Урагш засна» гэдэг стратеги нь өгөгдлийн бүтэц өөрчлөгдсөн үед л зөвтгөгдөнө.

Инцидент, асуудал, өөрчлөлт — ялгаа

Ойлголт	Тодорхойлолт	Зорилго
Incident	Үйлчилгээний тасалдал эсвэл чанарын доройтол	Аль болох хурдан сэргээх
Problem	Нэг ба хэд хэдэн инцидентийн үндсэн шалтгаан	Дахин давтагдахаас сэргийлэх
Change	Системд оруулж буй төлөвлөсөн өөрчлөлт	Эрсдэлийг удирдах

Инцидентийн үед зорилго нь шалтгааныг олох биш — үйлчилгээг сэргээх явдал. Шалтгааныг дараа нь олно.

Хүндрэлийн зэрэглэл ба хариу үйлдэл

Зэрэг	Тодорхойлолт	Хариу үйлдэл
SEV1	Үйлчилгээ бүрэн зогссон, өгөгдөл алдагдах эрсдэлтэй	Шууд дуудлага, удирдлагад мэдэгдэнэ, 24/7
SEV2	Гол функц ажиллахгүй, түр шийдэл байхгүй	Ажлын цагаар шууд, дуудлагатай
SEV3	Хэсэгчилсэн доройтол, түр шийдэл байгаа	Ажлын өдөрт шийднэ
SEV4	Бага нөлөөтэй, гоо сайхны асуудал	Төлөвлөгөөт ажилд оруулна

ТОДОРХОЙ БАЙХ ЁСТОЙ

Зэрэглэлийг **хэрэглэгчийн нөлөөгөөр** тодорхойлно, техникийн ноцтой байдлаар биш. Нэг сервер унасан ч хэрэглэгч мэдрээгүй бол SEV1 биш.

Инцидентийн үүргүүд

1. **Incident Commander**

   Шийдвэр гаргана, ажлыг зохицуулна. **Өөрөө техникийн засвар хийхгүй** — тэр нь анхаарлыг сарниулна.

2. **Operations Lead**

   Бодит техникийн үйлдлийг гүйцэтгэнэ (rollback, failover, тохиргоо).

3. **Communications Lead**

   Дотоод ба гадаад мэдээллийг хариуцна: status page, хэрэглэгч, удирдлага.

4. **Scribe**

   Цаг хугацааны дараалал, шийдвэр, ажиглалтыг бүртгэнэ — postmortem-ийн үндэс.

Жижиг багт нэг хүн хэд хэдэн үүрэг гүйцэтгэнэ. Гэхдээ **Incident Commander үргэлж тодорхой** байх ёстой.

Blameless postmortem

Тасалдлын дараа бичих дүн шинжилгээ. Гол зарчим: **хүнийг биш, системийг** судална.

Хэсэг	Агуулга
Хураангуй	Юу болсон, хэр удсан, хэнд нөлөөлсөн
Нөлөө	Хэрэглэгчийн тоо, алдагдсан гүйлгээ, error budget-ийн зарцуулалт
Цаг хугацааны дараалал	Илэрсэн, мэдэгдсэн, оношилсон, сэргээсэн цагууд
Үндсэн шалтгаан	Техникийн ба процессын шалтгаанууд
Юу сайн ажилласан	Хяналт барьсан уу, runbook тустай байсан уу
Үйл ажиллагааны төлөвлөгөө	Хариуцагч ба хугацаа бүхий тодорхой ажлууд

Асуултын хэлбэрийг өөрчлөх

Буруутгасан асуулт	Системийн асуулт
Хэн production дээр устгасан бэ?	Яагаад нэг команд production-д устгал хийж чадсан бэ?
Яагаад шалгаагүй юм бэ?	Яагаад автомат шалгалт байгаагүй вэ?
Яагаад мэдээгүй юм бэ?	Яагаад хяналт анхааруулга өгөөгүй вэ?
Яагаад буруу тохируулсан бэ?	Яагаад буруу тохиргоо хүчинтэй гэж тооцогдсон бэ?

Хүн буруутгагдахаас айвал алдаагаа нуудаг. Алдаа нуугдвал систем сурдаггүй.

MTBF, MTTR ба бэлэн байдал

```
Availability = MTBF / (MTBF + MTTR)

MTBF — эвдрэл хоорондын дундаж хугацаа (Mean Time Between Failures)
MTTR — сэргээх дундаж хугацаа  (Mean Time To Recovery)
MTTD — илрүүлэх дундаж хугацаа (Mean Time To Detect)
```

ОРЧИН ҮЕИЙН ОНЦЛОЛ

Уламжлалт хандлага **MTBF-ийг өсгөхийг** зорьдог байсан (эвдрэхгүй байх). Орчин үеийн хандлага **MTTR-ийг багасгахыг** зорьдог (хурдан сэргэх). Учир нь нарийн төвөгтэй системд эвдрэлийг бүрэн зайлуулах боломжгүй.

MTTR-ийг багасгах гурван арга: сайн **хяналт** (MTTD бага), бэлэн **runbook**, автомат **буцаалт**.

Нөөцлөлт: RPO ба RTO

Хэмжүүр	Асуулт	Утга
RPO	Хэдий хэмжээний өгөгдөл алдаж болох вэ?	Нөөцлөлтийн давтамжийг тодорхойлно
RTO	Хэдий хугацаанд сэргээх ёстой вэ?	Сэргээх архитектурыг тодорхойлно

Жишээ: `RPO = 1 цаг` бол нөөцлөлт цагт нэг удаа хийгдэнэ. `RTO = 4 цаг` бол сэргээх процесс 4 цагт багтах ёстой — үүнийг **бодитоор хэмжсэн** байх ёстой.

3-2-1 ДҮРЭМ

3 хуулбар · **2** өөр төрлийн зөөгч · **1** нь өөр байршилд. Орчин үеийн нэмэлт: **1** нь өөрчлөгдөшгүй (immutable) — ransomware-ээс хамгаалахын тулд.

Шалгаагүй нөөцлөлт бол нөөцлөлт биш

Нөөцлөлт байгаа эсэх нь чухал биш — СЭРГЭЭЖ чадах эсэх нь чухал.

Практикт хамгийн олон тохиолддог гамшиг бол: нөөцлөлт хийгдэж байсан ч сэргээх үед **ажиллахгүй** байх явдал.

- Нөөцлөлтийг тогтмол (сар бүр) **бодитоор сэргээж** үзнэ
- Сэргээх **хугацааг хэмжинэ** — RTO биелж байгаа эсэхийг батална
- Сэргээсэн өгөгдлийн **бүрэн бүтэн байдлыг** шалгана (checksum, түүврийн шалгалт)
- Сэргээх зааврыг **runbook** болгож, дадлагажуулна
- Нөөцлөлт **амжилтгүй** болсныг мэдэгдэх хяналт заавал байна

Ашиглалтаас гаргах

Хамгийн их орхигддог үе шат. Ашиглалтаас гараагүй хуучин систем нь **нөхөөс аваагүй, хяналтгүй, хариуцагчгүй** үлддэг тул халдлагын гол хаалга болдог.

1. **Хамаарлыг тогтоох**

   Энэ системээс хэн хамааралтай вэ? Хаана DNS бичлэг, интеграц үлдсэн вэ?

2. **Өгөгдлийг шилжүүлэх, архивлах**

   Хууль эрх зүйн хадгалалтын хугацааг тооцно.

3. **Урсгалыг зогсоох, ажиглах**

   Тодорхой хугацаанд унтраасан төлөвт үлдээж, хэн нэгэн хандаж байгаа эсэхийг харна.

4. **Найдвартай устгах**

   Диск, нөөцлөлт, cloud snapshot бүгд. Устгалыг баримтжуулна.

5. **Эрх, бүртгэлийг цэвэрлэх**

   Хэрэглэгчийн эрх, service account, API түлхүүр, гэрчилгээ, DNS.

ХЭСЭГ III

Automation-first

Toil бууруулах, дэд бүтцийг кодоор тодорхойлох, idempotency, GitOps, observability.

Automation-first гэж юу вэ

Automation-first нь «эхлээд гараар хийгээд, дараа нь автоматжуулъя» гэсэн дараалыг эргүүлж, **шинэ ажлыг эхнээс нь автоматжуулах** боломжтой хэлбэрээр зохион байгуулах хандлага юм.

Хоёр хандлагын ялгаа нь өртгийн муруйд оршино.

Хандлага	Эхний өртөг	10 дахь удаа	100 дахь удаа
Гараар	Бага	10× өртөг	100× өртөг + алдаа
Автоматжуулсан	Өндөр (3–5×)	Бараг тэг	Бараг тэг

ШИЙДВЭРИЙН ДҮРЭМ

Ажлыг **гурав дахь удаагаа** гараар хийж байвал энэ нь автоматжуулах дохио юм. Гэхдээ бүхнийг автоматжуулах нь зөв биш — жилд нэг удаа хийгддэг ажилд автоматжуулалт хэт үнэтэй.

Автоматжуулалтын таван шат

1 **Гараар**

Хүн SSH-ээр нэвтэрч тохируулна. Баримтжуулалт нь хүний толгойд.

1 **Баримтжуулсан**

Runbook бичигдсэн. Хүн дагана — гэхдээ хүн л гүйцэтгэнэ.

2 **Скрипт**

Скрипт бичигдсэн, хүн ажиллуулна. Алдаа багасна, гэхдээ хүн шаардлагатай.

3 **Тунхаглалт (declarative)**

Хүссэн **эцсийн төлөвийг** тодорхойлно, систем өөрөө нийцүүлнэ. Terraform, Ansible, Kubernetes.

4 **Өөрөө үйлчилгээ (self-service)**

Хэрэглэгч өөрөө хүсэлт өгч, систем автоматаар гүйцэтгэнэ. Инженер оролцохгүй.

Ихэнх байгууллага 1–2 дугаар шатанд байдаг. Зорилго нь 3 руу шилжиж, чухал урсгалуудыг 4 болгох явдал.

Idempotency — давтан ажиллуулах аюулгүй байдал

Idempotent үйлдэл нь нэг ч удаа, зуун удаа ажиллуулсан ч **ижил эцсийн төлөвт** хүргэдэг.

IDEMPOTENT БИШ — ДАВТАН АЖИЛЛУУЛБАЛ ЭВДЭРНЭ

```
echo '10.0.0.5 db.local' >> /etc/hosts     # мөр давхардсаар байна
useradd deploy                             # хоёр дахь удаа алдаа өгнө
iptables -A INPUT -p tcp --dport 22 -j ACCEPT   # дүрэм олширно
```

IDEMPOTENT — ХЭДЭН Ч УДАА АЖИЛЛУУЛЖ БОЛНО

```
- name: Ensure db host entry exists
  ansible.builtin.lineinfile:
    path: /etc/hosts
    line: "10.0.0.5 db.local"
    state: present

- name: Ensure deploy user exists
  ansible.builtin.user:
    name: deploy
    state: present
```

Infrastructure as Code

IaC нь дэд бүтцийг (сүлжээ, виртуал машин, өгөгдлийн сан, эрх) **эх код** хэлбэрээр тодорхойлж, хувилбарын хяналтад хадгалах арга юм.

Давуу тал	Практик утга
Хувилбарын хяналт	Хэн, хэзээ, юуг өөрчилснийг Git түүхээс харна
Хянан батлах	Өөрчлөлт нь pull request-ээр хянагдана
Давтагдац	Dev, staging, prod орчин ижил кодоос үүснэ
Баримтжуулалт	Код өөрөө үнэн эх сурвалж — wiki хуучрахгүй
Гамшгийн сэргээлт	Бүх дэд бүтцийг тэгээс дахин барих боломжтой
Урьдчилан харах	plan командаар өөрчлөлтийг гүйцэтгэхээс өмнө харна

IaC жишээ — Terraform

```
resource "aws_instance" "web" {
  count         = 3
  ami           = var.base_image
  instance_type = "t3.medium"

  tags = {
    Name        = "web-${count.index + 1}"
    Environment = var.environment
    Owner       = "platform-team"
    ManagedBy   = "terraform"
  }
}

resource "aws_security_group_rule" "https_only" {
  type              = "ingress"
  from_port         = 443
  to_port           = 443
  protocol          = "tcp"
  cidr_blocks       = ["0.0.0.0/0"]
  security_group_id = aws_security_group.web.id
}
```

`terraform plan` нь өөрчлөлтийг урьдчилан харуулна, `terraform apply` нь хэрэгжүүлнэ. Төлөв (state) файл нь бодит байдлыг хянана.

Тохиргооны удирдлага — Ansible

IaC нь дэд бүтцийг **үүсгэдэг**; тохиргооны удирдлага нь дотор нь юу байхыг **тодорхойлдог**.

```
- name: Configure web servers
  hosts: webservers
  become: true

  tasks:
    - name: Ensure nginx is installed
      ansible.builtin.package:
        name: nginx
        state: present

    - name: Deploy configuration
      ansible.builtin.template:
        src: nginx.conf.j2
        dest: /etc/nginx/nginx.conf
        owner: root
        mode: "0644"
        validate: "nginx -t -c %s"
      notify: reload nginx

  handlers:
    - name: reload nginx
      ansible.builtin.service:
        name: nginx
        state: reloaded
```

`validate` нь буруу тохиргоог **байршуулахаас өмнө** барина — үйлчилгээг унагахаас сэргийлнэ.

Immutable infrastructure

Хувиршгүй дэд бүтэц: ажиллаж буй сервер дээр өөрчлөлт хийхгүй. Өөрчлөлт хэрэгтэй бол **шинэ дүрс** (image) барьж, хуучныг солино.

Хувирамтгай (mutable)	Хувиршгүй (immutable)
Сервер дээр шинэчлэл, нөхөөс тавина	Шинэ image барьж, серверийг солино
Цаг хугацаанд серверүүд ялгаатай болно	Бүх сервер яг ижил
«Configuration drift» үүснэ	Drift үүсэх боломжгүй
Буцаах нь төвөгтэй	Өмнөх image-руу шилжих — хялбар
Оношилгоо хүнд (энэ сервер онцгой)	Оношилгоо хялбар (бүгд ижил)

CONFIGURATION DRIFT

Хугацаа өнгөрөх тусам гараар хийсэн жижиг өөрчлөлтүүдээс болж серверүүд хоорондоо ялгаатай болох үзэгдэл. «Яагаад зөвхөн web-03 дээр л ажиллахгүй байна вэ?» гэсэн асуултын гол шалтгаан.

GitOps

GitOps нь Git агуулахыг системийн хүссэн төлөвийн **цорын ганц үнэн эх сурвалж** болгож, автомат агент бодит төлөвийг түүнд нийцүүлдэг арга юм.

1. **Тунхаглалт**

   Хүссэн төлөв бүхэлдээ Git-д тодорхойлогдсон.

2. **Хувилбартай ба өөрчлөгдөшгүй**

   Түүх бүрэн хадгалагдана; өмнөх commit рүү буцаах нь буцаалт.

3. **Автомат татах**

   Агент өөрчлөлтийг өөрөө татаж хэрэгжүүлнэ — гараар `apply` хийхгүй.

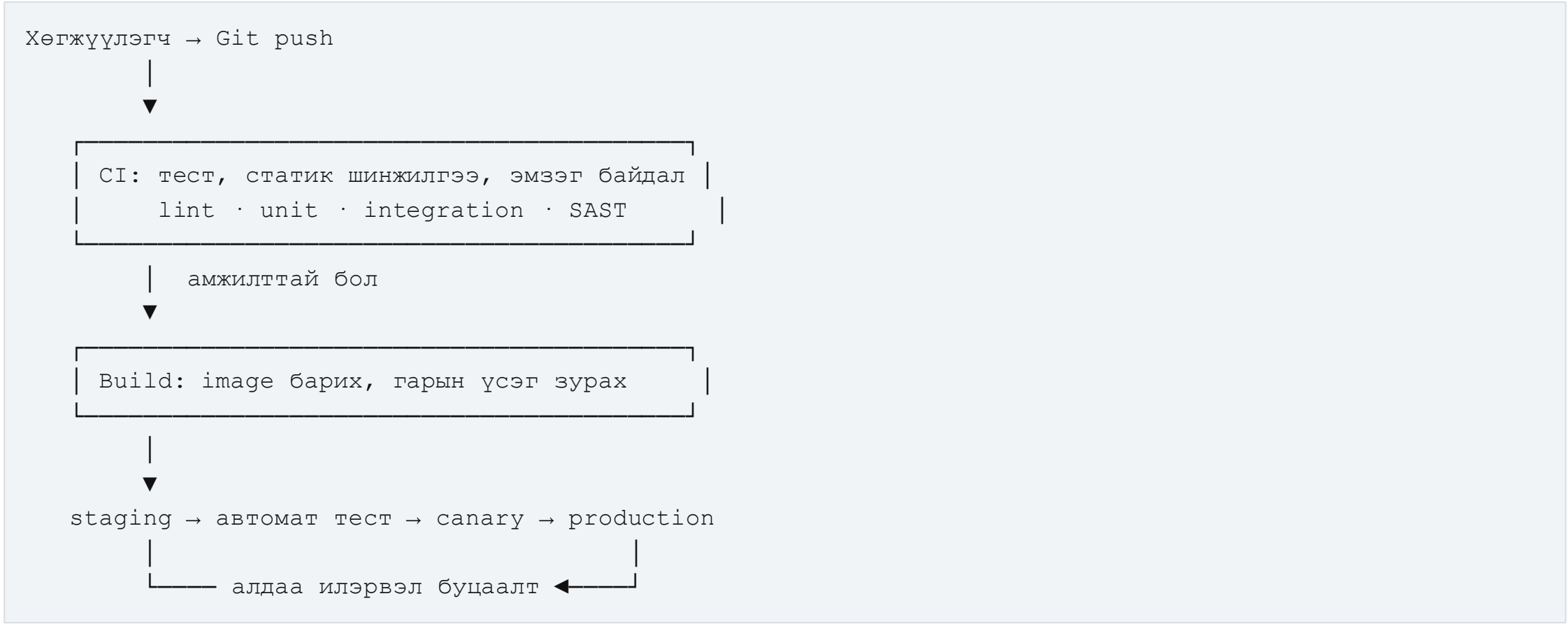
4. **Тасралтгүй нийцүүлэлт**

   Бодит төлөв зөрвөл агент автоматаар засна (drift-ийг арилгана).

АЮУЛГҮЙ БАЙДЛЫН ДАВУУ ТАЛ

Инженерүүд production рүү **шууд хандах эрхгүй** болно — бүх өөрчлөлт Git-ээр дамжина. Энэ нь аудит, хянан батлах, буцаалтыг нэг дор шийднэ.

CI/CD урсгал



Аюулгүй байдлын шалгалт (SAST, хамаарлын сканнер, нууц илрүүлэлт) нь урсгалын **эхэнд** байрлана — эцэст биш.

Хяналтын гурван багана

Багана	Юуг хариулдаг вэ	Жишээ хэрэгсэл
Metrics	Юу болж байна вэ? Хэр их вэ?	Prometheus, Grafana
Logs	Яг юу болсон бэ? Дэлгэрэнгүй	Loki, Elasticsearch
Traces	Хүсэлт хаана удсан бэ?	Jaeger, OpenTelemetry

MONITORING БА OBSERVABILITY

Monitoring нь урьдчилан мэдэж байсан асуултад хариулна («CPU 90% давсан уу?»). **Observability** нь урьдчилан бодоогүй асуултад хариулах чадвар («Яагаад зөвхөн Улаанбаатарын хэрэглэгчид удаан байна вэ?»).

Дөрвөн алтан дохио

1. **Latency — саатал**

   Хүсэлт хэр удаж байна вэ? Амжилттай ба амжилтгүй хүсэлтийг **тусад нь** хэмжинэ.

2. **Traffic — урсгал**

   Хэр их эрэлт байна вэ? Хүсэлт/сек, холболтын тоо.

3. **Errors — алдаа**

   Амжилтгүй хүсэлтийн хувь. Тод (500) ба далд (буруу агуулга) алдааг хоёуланг.

4. **Saturation — дүүрэлт**

   Систем хэр дүүрсэн бэ? CPU, санах ой, дискний I/O, холболтын сан.

Эдгээр дөрөв нь үйлчилгээний эрүүл мэндийг тодорхойлох хамгийн бага багц юм.

Сайн анхааруулга бичих

Муу анхааруулга	Сайн анхааруулга
CPU 90% давсан	Хүсэлтийн 5% нь алдаа буцааж байна
Диск 80% дүүрсэн	Одоогийн өсөлтөөр 4 цагийн дараа дүүрнэ
Сервис дахин эхэлсэн	Хэрэглэгчийн нэвтрэлт 10 минут амжилтгүй
Хариу үйлдэл шаардахгүй	Тодорхой runbook-той холбогдсон

ГОЛ ДҮРЭМ

Анхааруулга нь **шинж тэмдэгт** (symptom) суурилсан байх ёстой — шалтгаанд биш. Мөн шөнө дунд хүнийг сэрээж байгаа бол **тухайн хүн ямар нэг зүйл хийж чадах** ёстой.

Автоматжуулалтын эрсдэл

Автоматжуулалт нь алдааг **арилгадаггүй** — алдааны **хурд ба хамрах хүрээг** өсгөдөг.

Эрсдэл	Хамгаалалт
Blast radius — нэг алдаа бүх серверт	Үе шаттай гаргалт, бүсчлэл, canary
Автомат буруу залруулга	Автомат үйлдэлд хязгаар тавих, батлах алхам
Хэт итгэх — хүн шалгахаа больно	plan/diff-ийг заавал хянана
Нууц мэдээлэл кодод орох	Нууцын сан, урьдчилан скан
Скрипт хуучрах, хэн ч ойлгохгүй	Тест, эзэмшигч, тогтмол хяналт

Гараар хийвэл нэг серверийг эвдэнэ; автоматаар хийвэл мянган серверийг гурван секундэд эвдэнэ.

Хэрэгслийн газрын зураг

Зориулалт	Түгээмэл хэрэгслүүд
Хувилбарын хяналт	Git, GitHub / GitLab
Дэд бүтцийн код	Terraform, OpenTofu, Pulumi, CloudFormation
Тохиргооны удирдлага	Ansible, Salt, Puppet, Chef
Дүрс барих	Packer, Docker, Buildah
Оркестрчлал	Kubernetes, Nomad, Docker Compose
CI/CD	GitHub Actions, GitLab CI, Jenkins, Argo CD
Хяналт	Prometheus, Grafana, Loki, OpenTelemetry, Zabbix
Нууцын удирдлага	HashiCorp Vault, SOPS, cloud secret manager
Аюулгүй байдал	Trivy, OpenSCAP, Falco, Wazuh

Хэрэгслүүд 3–5 жил тутам солигдоно; **зарчим нь солигддоггүй**. Хэрэгслийг биш, шийдэж буй асуудлыг сур.

ХЭСЭГ IV

Security-by-design

Аюулгүй байдлыг эцэст нэмдэг давхарга биш, загварын эхний шийдвэр болгох.

Security-by-design гэж юу вэ

Security-by-design нь аюулгүй байдлын хяналтыг систем ашиглалтад орсны дараа нэмдэг биш, **шаардлага ба загварын үе шатнаас** эхлэн суулгах хандлага юм.

Хойшлуулсан хандлага	Загварт суулгасан хандлага
Аудитын өмнө яаран засварлана	Шаардлагад тодорхойлогдсон байна
Аюулгүй байдал = саад	Аюулгүй байдал = өгөгдмөл байдал
Засварын өртөг өндөр	Загварын шийдвэр — нэмэлт өртөггүй
Хяналт нь тойрч гарах боломжтой	Хяналт нь урсгалын салшгүй хэсэг

Аюулгүй байдлыг эцэст нь нэмэх нь барилга барьсны дараа суурийг нь солихтой адил.

CIA гурвалжин

C **Confidentiality — Нууцлал**

Мэдээлэлд зөвхөн эрх бүхий этгээд хандана. Хэрэгсэл: шифрлэлт, эрхийн хяналт, сегментчлэл.

I **Integrity — Бүрэн бүтэн байдал**

Мэдээлэл зөвшөөрөлгүй өөрчлөгдөхгүй. Хэрэгсэл: checksum, гарын үсэг, хувилбарын хяналт, аудит.

A **Availability — Бэлэн байдал**

Эрх бүхий хэрэглэгч хэрэгтэй үедээ хандана. Хэрэгсэл: нөөцлөлт, redundancy, DDoS хамгаалалт.

ТЭНЦВЭР

Гурав нь ихэвчлэн **хоорондоо зөрчилддөг**. Хамгийн нууцлалтай систем нь хэн ч ашиглаж чадахгүй систем. Инженерийн ажил бол тэнцвэрийг зөв сонгох явдал.

Хамгаалалтын үндсэн зарчмууд

Зарчим	Утга	Практик хэрэгжилт
Least privilege	Хамгийн бага шаардлагатай эрх	RBAC, түр эрх, sudo дүрэм
Separation of duties	Нэг хүн бүх алхмыг хийхгүй	Хянан батлах, дөрвөн нүдний дүрэм
Defense in depth	Олон давхар хамгаалалт	Галт хана + hardening + IDS + аудит
Fail secure	Эвдрэхдээ хаалттай төлөвт	Эрх шалгагч унавал хандалт хориглоно
Secure defaults	Өгөгдмөл тохиргоо аюулгүй	Бүх порт хаалттай, дараа нь нээнэ
Minimize attack surface	Хэрэггүй бүхнийг унтраах	Шаардлагагүй үйлчилгээ, багц устгах
Complete mediation	Хандалт бүрийг шалгах	Кэшлэсэн эрхэд найдахгүй

Least privilege — практикт

ТҮГЭЭМЭЛ АЛДАА

- Бүх инженерт production дээр `root` эрх өгөх
- Програм өөрөө `root`-оор ажиллах
- Service account-д «бүх зүйл» хийх эрх өгөх
- Түр өгсөн эрх байнгын болж үлдэх (**privilege creep**)

ЗӨВ ХАНДЛАГА

- Үүрэгт суурилсан эрх (**RBAC**) — хувь хүнд биш, үүрэгт эрх онооно
- **Just-in-time** эрх — шаардлагатай үед, тодорхой хугацаагаар
- Тасалдлын үеийн **break-glass** данс — тусад нь, бүртгэл ба анхааруулгатай
- Эрхийн **тогтмол хяналт** (улирал тутам) — хэрэглэгдэхгүй эрхийг хураах

Zero Trust

Zero Trust нь «дотоод сүлжээ найдвартай» гэсэн таамаглалыг **бүрэн орхиж**, хандалт бүрийг тухай бүрд нь баталгаажуулах загвар юм.

Уламжлалт (периметр)	Zero Trust
Сүлжээний байрлалаар итгэнэ	Байршилд огт итгэхгүй
VPN-ээр орсон бол дотоод хэрэглэгч	Хандалт бүрд баталгаажуулна
Хатуу гадна, зөөлөн дотор	Хаа сайгүй ижил хатуу
Нэг нэвтрэлт = бүх нөөц	Нөөц бүрд тусдаа эрх

ГУРВАН ТУЛГУУР

Ил баталгаажуулалт (хэрэглэгч, төхөөрөмж, контекст) · **Хамгийн бага эрх** (JIT, хугацаатай) · **Эвдэрсэн гэж үзэх** (сегментчлэл, бүх зүйлийг бүртгэх).

Танилт ба эрх олголт

Ойлголт	Асуулт	Хэрэгсэл
Authentication	Чи хэн бэ?	Нууц үг, MFA, гэрчилгээ, SSH түлхүүр
Authorization	Чи юу хийж болох вэ?	RBAC, ABAC, бодлого
Accounting / Audit	Чи юу хийсэн бэ?	Бүртгэл, аудитын лог, SIEM

- **MFA** нь нууц үг алдагдсан үед хамгаалах хамгийн үр дүнтэй ганц арга хэмжээ
- **SSO** нь дансны тархалтыг бууруулж, ажлаас гарахад эрхийг нэг дор хураах боломж өгнө
- **Service account** нь хүний данснаас тусдаа, эрх нь хязгаарлагдсан, түлхүүр нь эргэлддэг байна
- Ажилтан гарахад эрхийг **хураах процесс** автоматжсан байх ёстой

Нууц мэдээллийн удирдлага

ХЭЗЭЭ Ч ХИЙЖ БОЛОХГҮЙ

Нууц үг, API түлхүүр, гэрчилгээний хувийн түлхүүрийг **эх кодын агуулахад** хадгалах. Git-ийн түүхээс устгах нь маш төвөгтэй — commit хийсэн даруйд тэр нууц **алдагдсан** гэж үзнэ.

Шаардлага	Хэрэгжүүлэлт
Төвлөрсөн хадгалалт	Vault, cloud secret manager
Шифрлэгдсэн байх	Хадгалалт ба дамжуулалтын үед
Хандалт бүртгэгдэх	Хэн, хэзээ, ямар нууцад хандсан
Эргэлт (rotation)	Тогтмол хугацаанд автоматаар шинэчлэх
Хамгийн бага эрх	Үйлчилгээ өөрт хэрэгтэй нууцад л хандана
Урьдчилан илрүүлэх	Commit хийхийн өмнө нууц сканнер ажиллана

Hardening ба secure defaults

Hardening нь системийн өгөгдмөл тохиргоог аюулгүй болгож, халдлагын гадаргууг багасгах ажил юм.

- Шаардлагагүй **үйлчилгээ, багц, порт** унтраах
- SSH: нууц үгээр нэвтрэхийг хааж, **түлхүүр** ашиглах; `root` шууд нэвтрэхийг хориглох
- Автомат **аюулгүй байдлын нөхөөс** идэвхжүүлэх
- **Галт ханын өгөгдмөл** дүрэм: бүгдийг хориглоод шаардлагатайг нээх
- Файлын эрх, `umask`, SELinux/AppArmor тохируулах
- Хугацаа синхрончлол (NTP) — аудитын лог утга учиртай байхын тулд
- **CIS Benchmark** зэрэг эталонтой тулган шалгах (OpenSCAP, Lynis)

Hardening-ийг гараар биш, **тохиргооны кодоор** хийнэ — тэгвэл шинэ сервер бүр аюулгүй төлөвтэй үүснэ.

Сүлжээний хамгаалалт

Давхарга	Хяналт
Сегментчлэл	VLAN, дэд сүлжээ, аюулгүй байдлын бүлэг; DMZ тусгаарлалт
Микросегментчлэл	Ажлын ачаалал бүрийн хооронд бодлого — «дотор бүгд нээлттэй» биш
Шифрлэлт	TLS 1.2+; дотоод урсгалыг ч шифрлэх (mTLS)
Алсын хандалт	VPN эсвэл zero-trust proxy; MFA заавал
Ирмэгийн хамгаалалт	WAF, DDoS хамгаалалт, хурдны хязгаарлалт
Илрүүлэлт	IDS/IPS, урсгалын бүртгэл (NetFlow), сонсох портын хяналт
DNS	DNSSEC, шүүлт, дотоод бүсийн тусгаарлалт

Эмзэг байдал ба нөхөөсийн менежмент

1. **Бүртгэл (inventory)**

   Юу байгааг мэдэхгүй бол хамгаалж чадахгүй. Хөрөнгийн автомат бүртгэл заавал.

2. **Скан**

   Систем ба хамаарлын эмзэг байдлыг тогтмол шалгана (Trivy, OpenVAS).

3. **Эрэмбэлэх**

   CVSS оноо + **бодит нөлөө** (ил гарсан уу, ашиглагдаж байна уу).

4. **Засах**

   Тохиролцсон хугацаанд: critical — 7 хоног, high — 30 хоног гэх мэт.

5. **Батлах**

   Засварыг дахин сканаар батална — тайланд найдахгүй.

ПРАКТИК АНХААРАЛ

Эмзэг байдлын дийлэнх нь өөрийн кодод биш, **гуравдагч талын хамаарлууд** дотор байдаг. Тиймээс хамаарлын сканыг CI урсгалд заавал оруулна.

Бүртгэл ба аудит

- **Юуг бүртгэх:** нэвтрэлт (амжилттай ба амжилтгүй), эрхийн өөрчлөлт, тохиргооны өөрчлөлт, өгөгдөлд хандалт, аюулгүй байдлын үйл явдал
- **Хаана хадгалах:** төвлөрсөн, **зөвхөн нэмэгддэг** (append-only) хадгалалт — халдагч ул мөрөө устгаж чадахгүй
- **Хэр удаан:** хууль эрх зүйн шаардлагаас хамаарна (ихэвчлэн 6–12 сар, зарим салбарт олон жил)
- **Юуг бүртгэхгүй:** нууц үг, токен, картын дугаар, хувийн эмзэг өгөгдөл
- **Хугацаа:** бүх систем NTP-ээр синхрончлогдсон байх — эс тэгвэл дараалал тогтоох боломжгүй

ПРАКТИК ШАЛГУУР

«Хэн, хэзээ, юуг, хаанаас, ямар үр дүнтэй» — энэ таван асуултад лог хариулж чадаж байвал аудитын шаардлагыг хангаж байна гэсэн үг.

Threat modeling — STRIDE

Загварын үе шатанд «энэ системийг хэн, яаж эвдэж болох вэ?» гэсэн асуултыг **системтэйгээр** тавих арга.

Заналхийлэл	Утга	Хамгаалалт
Spoofing	Өөрийг өөр хүн мэт харуулах	Хүчтэй танилт, MFA
Tampering	Өгөгдөл, кодыг өөрчлөх	Гарын үсэг, checksum, эрх
Repudiation	Хийсэн үйлдлээ үгүйсгэх	Аудитын лог, гарын үсэг
Information disclosure	Мэдээлэл задрах	Шифрлэлт, эрхийн хяналт
Denial of service	Үйлчилгээг зогсоох	Хурдны хязгаар, масштаб, DDoS хамгаалалт
Elevation of privilege	Эрхээ өсгөх	Least privilege, тусгаарлалт, hardening

Аюулгүй байдлын инцидентийн хариу үйлдэл

1. **Бэлтгэл**

   Төлөвлөгөө, үүрэг, холбоо барих жагсаалт, хэрэгсэл — **инцидентээс өмнө** бэлэн.

2. **Илрүүлэлт ба шинжилгээ**

   Дохиог үнэлэх, хамрах хүрээг тогтоох, нотлох баримт хадгалах.

3. **Хязгаарлалт**

   Тархалтыг зогсоох: сегментчлэх, эрх хураах, холболт таслах.

4. **Устгал**

   Халдагчийн ул мөр, хортой код, ашигласан эмзэг байдлыг арилгах.

5. **Сэргээлт**

   Цэвэр нөөцлөлтөөс сэргээх, хяналт эрчимжүүлэх, үе шаттай нээх.

6. **Сургамж**

   Postmortem, хяналтын сайжруулалт, төлөвлөгөөг шинэчлэх.

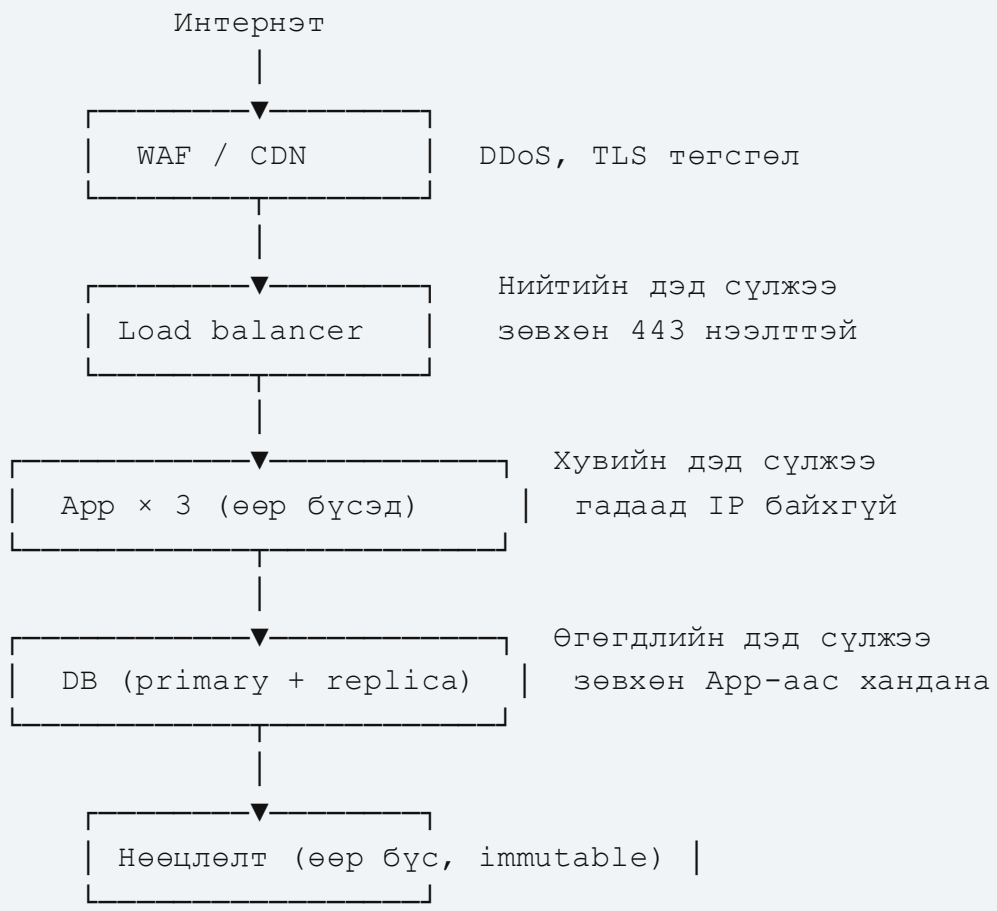
Ердийн тасалдлаас гол ялгаа: **нотлох баримтыг хадгалах** шаардлага. Яаран дахин суулгах нь мөрдөн байцаалтын боломжийг устгана.

ХЭСЭГ V

Практик нэгтгэл

Лавлах архитектур, runbook, хэмжүүрүүд, түгээмэл алдаа.

Лавлах архитектур — жижиг вэб үйлчилгээ



Энэ архитектурт шингэсэн зарчмууд

Зарчим	Хаана илэрч байна
Defense in depth	WAF → LB → SG → hardening → аудит
Least privilege	DB зөвхөн App-аас; App-д гадаад IP байхгүй
Minimize attack surface	Зөвхөн 443 порт нээлттэй
Redundancy	App × 3, өөр бүсэд; DB replica
Fail secure	SG-ийн өгөгдмөл — бүх урсгал хориотой
Recoverability	Өөр бүс дэх өөрчлөгдөшгүй нөөцлөлт
Automation	Бүх бүрэлдэхүүн IaC-ээр тодорхойлогдсон

Runbook — юу байх ёстой вэ

Хэсэг	Агуулга
Гарчиг ба хамрах хүрээ	Ямар нөхцөлд энэ runbook хэрэглэгдэх вэ
Шинж тэмдэг	Ямар анхааруулга, ямар алдааны мессеж
Нөлөөний үнэлгээ	Хэнд, хэр зэрэг нөлөөлж байна
Оношилгооны алхмууд	Юуг эхлээд шалгах — тодорхой команд
Хамгаалалтын арга	Хэрэглэгчид нөлөөг бууруулах түр шийдэл
Засварын алхмууд	Гүйцэтгэх командууд, хүлээгдэх үр дүн
Батлах	Засагдсаныг яаж баталгаажуулах вэ
Эскалаци	Хэзээ, хэнд дамжуулах вэ (нэр, суваг)

Сайн runbook нь **шөнийн 3 цагт, нойрмог хүн** дагаж чадахуйц тодорхой байх ёстой.

DORA хэмжүүрүүд

Багийн гүйцэтгэлийг үнэлэх дөрвөн салбарын стандарт хэмжүүр.

Хэмжүүр	Утга	Өндөр гүйцэтгэлтэй баг
Deployment frequency	Хэр олон удаа production-д гаргадаг вэ	Өдөрт олон удаа
Lead time for changes	Commit-оос production хүртэлх хугацаа	Нэг өдрөөс бага
Change failure rate	Гаргалтын хэдэн хувь нь асуудал үүсгэдэг вэ	15%-иас бага
Time to restore	Тасалдлаас сэргээх хугацаа	Нэг цагаас бага

ГОЛ ДҮГНЭЛТ

Судалгаанууд харуулснаар **хурд ба тогтвортой байдал зөрчилддөггүй** — өндөр гүйцэтгэлтэй багууд хурдан ч гаргадаг, найдвартай ч байдаг. Учир нь автоматжуулалт хоёуланг нь нэгэн зэрэг сайжруулдаг.

Түгээмэл алдаанууд

Алдаа	Үр дагавар	Шийдэл
Нөөцлөлтийг шалгадаггүй	Гамшгийн үед сэргээж чадахгүй	Сар бүр бодит сэргээлт
Гараар хийсэн тохиргоо	Configuration drift, давтагдахгүй	IaC ба тохиргооны удирдлага
Хэт олон анхааруулга	Alert fatigue — жинхэнэ дохиог алддаг	Symptom-based, тогтмол цэвэрлэгээ
Бүгдэд root эрх	Нэг алдаа бүх системд	RBAC, JIT эрх
Нууц кодын агуулахад	Түлхүүр алдагдана	Vault, pre-commit скан
Ашиглалтаас гаргаагүй систем	Нөхөөсгүй халдлагын хаалга	Lifecycle-ийн 7-р шат
Postmortem-гүй	Ижил тасалдал давтагдана	Blameless postmortem заавал
Баримтжуулалт хуучирсан	Runbook төөрөгдүүлнэ	Код өөрөө баримт; тогтмол хяналт

Хиймэл оюун ба системийн удирдлага

Сайн хийдэг	Найдваргүй хийдэг
Скрипт, Ansible role-ийн загвар гаргах	Таны сүлжээний топологийг таах
Алдааны мессежийг тайлбарлах	Аюулгүй байдлын шийдвэрийг эцэслэх
Лог, тохиргоог задлан шинжлэх	Эрхийн загварыг зөв тогтоох
Баримтжуулалт, runbook бичих	Production дээрх нөлөөг үнэлэх
Regex, томьёо гаргах	Хуучирсан, аюултай зөвлөмж өгөхгүй байх

ОНЦГОЙ АНХААРУУЛГА

Хиймэл оюунаас гарсан командыг **ойлголгүйгээр production дээр ажиллуулах** нь энэ мэргэжлийн хамгийн эрсдэлтэй зан үйл. `rm`, `iptables -F`, `terraform destroy` зэрэг командын үр дагавар эргэлт буцалтгүй байж болно.

Гол санаанууд

- Орчин үеийн администратор бол засварладаг хүн биш, **засварлах шаардлагагүй систем зохион байгуулдаг инженер**
- Үйлчилгээ нь **долоон үе шаттай мөчлөгтэй** — ашиглалтаас гаргах шат ч үүнд багтана
- **SLI/SLO/error budget** нь найдвартай байдлыг хэлэлцээ биш, **тоон шийдвэр** болгоно
- Инцидентийн үед зорилго нь шалтгаан олох биш, **сэргээх**; шалтгааныг postmortem-д олно
- **Toil** нь өсөлттэй шугаман өсдөг ажил — 50%-иас хэтэрвэл системийн асуудал
- **Idempotency** нь автоматжуулалтын суурь шинж — давтан ажиллуулж болохгүй бол хэрэглэгдэхгүй
- **IaC ба GitOps** нь хувилбарын хяналт, хянан батлах, аудит, буцаалтыг нэг дор шийднэ
- **Аюулгүй байдал** нь эцсийн давхарга биш, загварын эхний шийдвэр
- **Least privilege, defense in depth, secure defaults** — гурван суурь зарчим
- Шалгаагүй нөөцлөлт бол **нөөцлөлт биш**

Гурван өгүүлбэрээр

Гараар хоёр дахь удаагаа хийж байгаа бүхэн бол ирээдүйн автоматжуулалтын жагсаалт.

Хэмжигдээгүй найдвартай байдал бол зөвхөн итгэл найдвар.

Аюулгүй байдлыг загварт суулгах нь эцэст нэмэхээс үргэлж хямд.

Нэр томьёо (1)

Нэр томьёо	Тодорхойлолт
SLI / SLO / SLA	Хэмжигдэхүүн / дотоод зорилт / хэрэглэгчтэй байгуулсан гэрээ
Error budget	SLO-оос үлдэх зөвшөөрөгдөх алдааны хэмжээ (100% − SLO)
Toil	Давтагдах, гараар хийгддэг, урт хугацааны үнэ цэнгүй ажил
MTTR / MTBF / MTTD	Сэргээх / эвдрэл хоорондын / илрүүлэх дундаж хугацаа
RPO / RTO	Зөвшөөрөгдөх өгөгдлийн алдагдал / сэргээх хугацаа
Idempotency	Давтан ажиллуулахад ижил эцсийн төлөвт хүргэх шинж
IaC	Дэд бүтцийг эх код хэлбэрээр тодорхойлох арга
GitOps	Git-ийг хүссэн төлөвийн цорын ганц эх сурвалж болгох арга

Нэр томьёо (2)

Нэр томьёо	Тодорхойлолт
Configuration drift	Серверүүд цаг хугацаанд хоорондоо ялгаатай болох үзэгдэл
Immutable infrastructure	Ажиллаж буй системийг өөрчлөхгүй, солих загвар
Blast radius	Нэг алдааны нөлөөлөх хамрах хүрээ
Observability	Урьдчилан бодоогүй асуултад хариулах чадвар
Least privilege	Хамгийн бага шаардлагатай эрхийг л олгох зарчим
Defense in depth	Олон бие даасан хамгаалалтын давхарга
Zero Trust	Сүлжээний байршилд итгэхгүй, хандалт бүрийг баталгаажуулах загвар
Blameless postmortem	Хүнийг биш системийг судалдаг тасалдлын дүн шинжилгээ

Өөрийгөө шалгах асуултууд

1. SysAdmin, SRE, DevOps, Platform Engineer-ийн үүргийн ялгааг тайлбарла.
2. Toil-ийн зургаан шинжийг нэрлэ. Ажил toil мөн эсэхийг яаж тогтоох вэ?
3. SLI, SLO, SLA гурвын ялгаа юу вэ? Аль нь хууль зүйн үр дагавартай вэ?
4. 99.99% бэлэн байдал сард хэдэн минут тасалдал зөвшөөрөх вэ?
5. Error budget юунд хэрэгтэй вэ? Дуусвал ямар шийдвэр гаргах вэ?
6. Инцидент, асуудал, өөрчлөлт гурвын ялгаа юу вэ?
7. Тасалдлын үед хамгийн эхний зорилго юу байх ёстой вэ? Яагаад?
8. Blameless postmortem-ийн гол зарчим юу вэ? Асуултын хэлбэрийг яаж өөрчлөх вэ?
9. RPO ба RTO-г тодорхойл. 3-2-1 дүрмийг тайлбарла.
10. Idempotency гэж юу вэ? Idempotent биш командын жишээ гарга.
11. Configuration drift яагаад үүсдэг вэ? Immutable загвар яаж шийддэг вэ?
12. GitOps-ийн дөрвөн зарчмыг нэрлэ. Аюулгүй байдлын ямар давуу талтай вэ?
13. Дөрвөн алтан дохиог нэрлэ. Сайн анхааруулгын шалгуур юу вэ?
14. Least privilege-ийн практик зөрчлүүдийг нэрлэ. Privilege creep гэж юу вэ?
15. Zero Trust нь уламжлалт периметр загвараас юугаараа ялгаатай вэ?
16. Нууц мэдээллийг Git-д commit хийвэл яагаад устгахад хангалтгүй вэ?
17. STRIDE-ийн зургаан заналхийллийг нэрлэ.
18. DORA-гийн дөрвөн хэмжүүрийг нэрлэ. Хүрд ба тогтвортой байдал зөрчилддөг үү?

Лабораторийн ажлын чиглэл

- Виртуал машин дээр Linux суулгаж, **hardening checklist**-ээр тохируулах (SSH түлхүүр, галт хана, шаардлагагүй үйлчилгээ)
- Ижил тохиргоог **Ansible role** болгож бичих; хоёр удаа ажиллуулж **idempotency**-г батлах
- Гар аргаар нэг сервер, Ansible-ээр гурван сервер тохируулж, **хугацаа ба алдааг** харьцуулах
- Хоёр серверийн тохиргоог тулгаж **configuration drift**-ийг илрүүлэх
- **Prometheus + Grafana** суулгаж, дөрвөн алтан дохиог хэмжих dashboard хийх
- Утга учиртай **анхааруулгын дүрэм** бичих; санаатай ачаалал үүсгэж дуудуулах
- Нэг үйлчилгээнд **SLI/SLO** тодорхойлж, 7 хоногийн error budget-ийг тооцоолох
- **Нөөцлөлт хийж, өөр машин дээр сэргээх**; RTO-г бодитоор хэмжих
- Санаатай тасалдал үүсгэж, **runbook** дагаж сэргээх; **postmortem** бичих
- Нууцыг Git-д санамсаргүй оруулж, **gitleaks**-ээр илрүүлэх; pre-commit hook суулгах
- Жижиг үйлчилгээний **threat model** (STRIDE) бичиж, хамгаалалт санал болгох

Дараагийн хичээл

Хичээл 2 — Linux системийн үндэс: файлын систем, процесс, эрх, systemd, багцын удирдлага.

Энэ лекцээр **юуг, яагаад** хийдгийг үзлээ. Дараагийн хичээлүүдээс эхлэн **яаж** хийхийг практикаар судална.

БЭЛТГЭЛ

- Виртуал машин (VirtualBox эсвэл KVM) суулгаж, Linux-ийн сүүлийн LTS хувилбарыг бэлтгэх
- Git суулгаж, өөрийн агуулах үүсгэх — бүх лабораторийн код тэнд хадгалагдана
- SSH түлхүүрийн хос үүсгэж, нууц үгээр нэвтрэхийг унтраах